IES Leonardo Da Vinci
Albacete

WattSphere

Proyecto Final del Ciclo Formativo de Grado Superior:
Desarrollo de Aplicaciones Multiplataforma

Familia Profesional de Informática y Comunicaciones
- Curso 2025-2026 –

Alumno/s: Iván Sánchez Bonilla

Tutor:

Índice

Índice 2
1. Introducción 3
2. Justificación 4
3. Aplicación en el marco tecnológico actual 5
4. Desarrollo del proyecto 7
5. Metodología y planificación 14
6. Conclusiones y Líneas futuras 16
7. Anexos 17
8. Bibliografía y webgrafía 18
9. Agradecimientos 19

1. Introducción

El presente proyecto consiste en el desarrollo de una aplicación móvil nativa para Android. Se trata de un videojuego que fusiona el género **"Incremental"** (*idle game*) con mecánicas de **progresión Roguelike** y un sistema de **Prestigio (Rebirth)**, bajo una temática de gestión y evolución energética.

El objetivo principal es ofrecer una experiencia de crecimiento constante donde el jugador gestiona recursos para escalar la producción de energía desde niveles básicos hasta escalas galácticas. La innovación de este título reside en dos mecánicas clave:

- **Mecánica Roguelike:** Al alcanzar hitos específicos, el jugador debe elegir entre tres mejoras aleatorias (*perks*). Esta decisión es estratégica y varía en cada sesión, determinando el estilo de producción de esa partida.
- **Sistema de Prestigio (Rebirth):** Permite reiniciar el progreso actual a cambio de una moneda especial permanente, otorgando bonificadores masivos que permiten superar niveles antes inalcanzables.

Niveles de energía diseñados:

- **Iniciales:** Paneles solares, eólica, hidroeléctrica.
- **Intermedios:** Biomasa, geotérmica, nuclear, mareomotriz.
- **Avanzados:** Esferas de Dyson, terraformación y civilizaciones tipo II-III (Kardashev).

Abstract

This project focuses on the development of a mobile video game for Android that combines "Incremental" mechanics with "Roguelike" progression and a "Rebirth" (Prestige) system. The main theme revolves around energy generation, evolving from basic renewable sources to theoretical sci-fi concepts like Dyson Spheres. A key feature is the strategic decision-making: upon reaching milestones, players choose one of three random perks to boost their progress. The application is built with Java and Android Studio, utilizing Google Firebase as a Backend-as-a-Service (BaaS) for real-time data persistence and authentication.

2. Justificación

La elección de este proyecto no solo responde al interés por el desarrollo de videojuegos, sino a un análisis detallado de las mecánicas que dominan el mercado móvil actual. La combinación de elementos *Incremental*, *Roguelike* y sistemas de *Prestigio* se justifica por los siguientes puntos:

2.1. Psicología del jugador y retención (Engagement)

Los juegos de tipo "Idle" o incrementales basan su éxito en la gratificación instantánea. Sin embargo, suelen pecar de ser lineales. Al introducir el componente **Roguelike**, se rompe la monotonía: el jugador ya no solo espera a que los números suban, sino que debe tomar decisiones activas cada vez que se le presentan las tres opciones de mejora (*perks*). Esta incertidumbre sobre qué mejoras aparecerán en cada partida genera un ciclo de juego mucho más dinámico y estimulante.

Desde una perspectiva de diseño de producto, esta mezcla de géneros ataca dos perfiles de usuario distintos: el jugador casual, que busca sesiones cortas de administración, y el jugador "hardcore", que busca optimizar matemáticamente sus decisiones (theorycrafting). Al unir ambos mundos, el proyecto maximiza su público objetivo potencial, justificando su desarrollo como un producto comercialmente viable en la Google Play Store.

2.2. Rejugabilidad infinita mediante el sistema de Prestigio

Uno de los mayores retos en el desarrollo de software de entretenimiento es el "fin del contenido". El sistema de **Rebirth (Prestigio)** soluciona esto de manera técnica y creativa. Permite que el código base sea altamente eficiente: en lugar de crear infinitos niveles distintos, se diseña un sistema de escalado matemático donde el jugador repite el ciclo inicial pero con una potencia muy superior. Esto crea un bucle de retroalimentación donde el usuario siente que su tiempo invertido se traduce en un poder permanente (moneda de prestigio), incentivando el retorno diario a la aplicación.

2.3. Temática y concienciación

La temática energética permite aplicar conceptos reales (energías renovables frente a fósiles) y teóricos (física avanzada) de una forma lúdica. El progreso desde una placa solar hasta una Esfera de Dyson no solo es satisfactorio visualmente, sino que sirve como hilo conductor para una curva de dificultad lógica y comprensible para cualquier tipo de usuario.

3. Aplicación en el marco tecnológico actual

El desarrollo de este videojuego se sitúa en la intersección de varias tendencias tecnológicas clave que definen el ecosistema de aplicaciones móviles en 2025: las arquitecturas **BaaS (Backend-as-a-Service)**, el desarrollo nativo optimizado y los modelos de rejugabilidad basados en sistemas de prestigio.

3.1. Evolución hacia Arquitecturas Serverless (Firebase)

Tradicionalmente, el desarrollo de aplicaciones que requerían persistencia de datos implicaba la gestión de servidores físicos o virtuales, configuración de APIs REST y mantenimiento de bases de datos relacionales (como MySQL). En el marco tecnológico actual, se ha producido un cambio de paradigma hacia el **Serverless Computing**.

El uso de **Google Firebase** en este proyecto ejemplifica esta tendencia. Al delegar la infraestructura a Firebase, el desarrollo se centra en la lógica de negocio y la experiencia de usuario. Esta tecnología permite:

- **Sincronización en tiempo real:** Crucial para que el progreso generado "en segundo plano" se refleje instantáneamente al abrir la aplicación en cualquier dispositivo.
- **Escalabilidad elástica:** La infraestructura crece automáticamente según el número de usuarios, una característica vital en el mercado de apps móviles donde el éxito puede ser viral y repentino.

Esta arquitectura reduce drásticamente los costes de mantenimiento inicial (OpEx) y elimina la necesidad de configurar parches de seguridad en el sistema operativo del servidor, ya que es responsabilidad del proveedor (Google)

3.2. El auge de los modelos "Hybrid-Casual"

El mercado actual de videojuegos móviles está evolucionando del *Hyper-casual* (juegos muy simples) al **Hybrid-casual**. Este proyecto encaja perfectamente en esta tendencia al combinar:

- La accesibilidad de los juegos **Incrementales**.
- La profundidad estratégica de los **Roguelikes** (toma de decisiones).
- La progresión a largo plazo de los sistemas de **Prestigio**.

Desde un punto de vista tecnológico, implementar un motor de decisiones aleatorias (las 3 opciones de *perks*) y un sistema de reinicio con beneficios permanentes responde a lo que los usuarios actuales demandan: mecánicas simples para sesiones cortas, pero con una capa de complejidad que premie la fidelidad a largo plazo.

3.3. Integración de Inteligencia Artificial en el flujo de trabajo

Siguiendo las metodologías de desarrollo actuales, este proyecto integra el uso de **IA generativa** para la creación de *assets* visuales. Esto refleja cómo las pequeñas producciones o desarrolladores independientes pueden competir hoy en día en calidad visual, optimizando tiempos de diseño y permitiendo que el desarrollador se enfoque en la arquitectura técnica y la robustez del código.

4. Desarrollo del proyecto

4.1 Diseño de la infraestructura

La infraestructura se divide en dos capas principales conectadas mediante el SDK de Google:

- **Frontend (App Android):** Desarrollada en Java. Gestiona la interfaz, los cálculos de energía por segundo y el motor de selección aleatoria del roguelike.
- **Backend (Firebase):**
  - *Cloud Auth:* Gestiona el acceso de usuarios.
  - *Realtime Database / Firestore:* Almacena el estado de la partida (recursos, mejoras temporales y multiplicadores de prestigio).

Arquitectura y Ciclo de Vida: Para garantizar que la aplicación sea mantenible y escalable, se ha seguido el patrón de arquitectura **MVVM (Model-View-ViewModel)**.

- **Model:** Define la estructura de datos de los generadores (ID, coste base, coeficiente de crecimiento, producción base).
- **ViewModel:** Actúa como intermediario, manteniendo el estado de la UI y sobreviviendo a los cambios de configuración (como la rotación de pantalla). Aquí reside la lógica del GameLoop que actualiza los contadores de energía.
- **View:** Fragmentos y Actividades que observan los cambios en el ViewModel mediante LiveData para actualizar la interfaz gráfica en tiempo real sin bloquear el hilo principal.

Adicionalmente, se ha diseñado un algoritmo de "Tiempo Delta" para la gestión de la producción *offline*. Al cerrar la app, se guarda un **timestamp** en Firebase. Al abrirla de nuevo, el sistema calcula la diferencia: **(TiempoActual - TiempoGuardado) * ProducciónPorSegundo * Velocidad**, otorgando al jugador la energía generada durante su ausencia.

4.2 Implementación y configuración

- **Lenguaje:** Java (Lógica del juego, algoritmos de prestigio y selección de perks).
- **Entorno:** Android Studio (IDE principal y compilador APK).
- **Interfaz:** XML (Layouts adaptables para diferentes tamaños de pantalla).
- **Integración Cloud:** Uso del archivo google-services.json para vincular la app con el proyecto Firebase.

Algoritmos y Estructuras de Datos:

- **Gestión de Grandes Números:** Dado que los valores superan el límite de Long.MAX_VALUE (9 trillones), se ha implementado una clase utilitaria BigNumberHelper basada en java.math.BigInteger. Esta clase incluye métodos para formatear las cifras en notación de ingeniería (ej: 1K, 1M, 1B, 1T, 1aa, 1bb), facilitando la lectura por parte del usuario.
- **Algoritmo Roguelike Ponderado:** El sistema de selección de cartas no es puramente aleatorio. Se utiliza un algoritmo de selección ponderada (Weighted Random Selection). Cada mejora tiene un atributo de "rareza" (Común: 70%, Rara: 25%, Legendaria: 5%). El sistema genera un número aleatorio entre 0 y 100 y selecciona la carta correspondiente a ese rango, asegurando que las mejoras más poderosas sean estadísticamente menos frecuentes, manteniendo el equilibrio del juego.
- **Persistencia Local:** Se utiliza SharedPreferences para guardar progreso local en caso de que el usuario no quiera iniciar sesión.
- **Fundamentos Matemáticos y Algorítimicos:** El núcleo de la jugabilidad se basa en el equilibrio numérico. Dado que el objetivo es escalar la producción hasta niveles galácticos, se ha diseñado un modelo matemático que regula la curva de dificultad y la economía del juego.

  Función de Coste Exponencial

  Para evitar que el jugador adquiera mejoras de forma lineal e infinita, el coste de los generadores sigue una progresión geométrica estándar en la industria de los juegos incrementales. La fórmula implementada en el código Java es la siguiente:

  Costo(n) = Costo(base) * (r)^(n)

  Donde:

- Costo(n): Es el precio del siguiente edificio.
- Costo(base): Es el precio inicial definido en el Model.
- r: Es el coeficiente de crecimiento (growth coefficient), establecido en **1.15**.
- n: Es el número actual de edificios de ese tipo que posee el jugador.

  Esta fórmula asegura que el precio aumente un 15% con cada compra, obligando al usuario a buscar multiplicadores de eficiencia (Perks) o realizar un *Prestigio* cuando el coste supera su capacidad de producción logarítmica.

4.3 Administración y monitorización

La gestión del código se realiza a través de **Git y GitHub**. El seguimiento de la actividad de los usuarios y el estado de la base de datos se monitoriza directamente desde la **Consola de Firebase**, que ofrece analíticas de uso y control de errores en tiempo real.

La monitorización mediante **Firebase Crashlytics** es vital en este proyecto: permite detectar "NullPointerExceptions" o desbordamientos de memoria en dispositivos específicos, enviando informes detallados de la pila de errores (stack trace) que facilitan la corrección rápida de bugs sin necesidad de tener acceso físico al dispositivo del usuario.

4.4 Medidas de seguridad de la infraestructura

Para proteger la integridad del progreso de los usuarios y garantizar un entorno seguro, se han implementado las siguientes medidas:

- **Autenticación Segura**: El acceso al sistema de guardado en la nube está restringido mediante Firebase Auth y el protocolo OAuth 2.0 de Google. Esto asegura que solo el propietario de la cuenta pueda acceder a su nodo de datos específico.
- **Reglas de Seguridad de Firebase**: Se han definido reglas en la base de datos que impiden que un usuario lea o escriba en los datos de otro. La estructura de permisos valida que el auth.uid del solicitante coincida con el ID del nodo al que intenta acceder.
- **Persistencia y Validación Local**: Los datos sensibles que se manejan en SharedPreferences están vinculados al ciclo de vida de la aplicación. Además, el uso de un servicio (EnergiaService) ayuda a que el estado del juego sea consistente y menos propenso a errores de manipulación externa por parte del usuario.
- **Validacion de datos:** Las medidas de validacion implementadas en FirebaseService incluyen verificaciones multiples:

  Validacion de rangos: Los valores de energia nunca pueden ser negativos
  Validacion de velocidad: Solo se aceptan velocidades entre 1 y 10
  Validacion de estructura: Los niveles no pueden ser menores a 0 o mayores al maximoteorico
  Timestamp de integridad: Se guarda el timestamp de ultima actualizacion para detectar manipulaciones
  Estas validaciones se aplican tanto en el cliente (para UX) como en el servidor Firebase (para seguridad).

4.5 Manual de usuario

La aplicación está diseñada bajo el principio de "fácil de aprender, difícil de dominar". El flujo principal se divide en:

1. **Inicio de Sesión:** El usuario accede mediante su cuenta de Google. Si no hay conexión, el juego permite jugar localmente hasta que la sincronización sea posible.

2. **Pantalla Principal (Home):** Aquí se visualiza la energía acumulada (kWS). El usuario puede generar energía manualmente realizando "taps" sobre el generador central.

3. **Tienda de Estructuras:** El usuario gasta su energía para comprar paneles solares, turbinas eólicas y otras mejoras. Cada compra aumenta la generación automática de energía por segundo.

4. **Gestión y Ajustes:** El usuario puede cerrar sesión, restablecer todo el progreso borrando los datos locales y en Firebase y se puede consultar la versión de la aplicación, los créditos del juego y se puede acceder a una página de soporte.

5. Metodología y planificación

5.1 Metodología usada para el desarrollo del proyecto

Se ha empleado una metodología **Ágil (tipo Scrum)** por ciclos:

- **Sprint 1:** Lógica básica de clics y edificios.
- **Sprint 2:** Integración de Firebase y sistema de guardado.
- **Sprint 3:** Implementación del motor Roguelike y el sistema de Prestigio.

5.2 Planificación

- **Fase 1:** Diseño de mecánicas y estructura de la base de datos NoSQL.
- **Fase 2:** Programación de la interfaz de usuario y lógica de energía.
- **Fase 3:** Integración de Firebase Auth y Database.
- **Fase 4:** Fase de testeo y balanceo de multiplicadores.

5.3 Recursos técnicos y equipamiento necesarios

- Ordenador con Android Studio.
- Smartphone Android (físico o emulado).
- Consola de Google Firebase.
- IA Generativa para la creación de *assets* visuales (iconos de edificios energéticos).

5.4 Presupuesto

El coste es mínimo gracias al uso de herramientas gratuitas:

- **Software**: Android Studio y Java (Gratis).
- **Hosting**: Firebase Plan Spark (Gratis hasta cierto volumen de tráfico).
- **Diseño**: Recursos creados mediante IA y herramientas de diseño abierto.

5.5 Pruebas y resultados

Se han realizado pruebas para asegurar:

1. Que la moneda de prestigio persiste tras el borrado de edificios.
2. Que las 3 opciones roguelike se generan de forma aleatoria y no se repiten en exceso.
3. Que el guardado en la nube funciona correctamente tras cerrar la app.

Adicionalmente, se han llevado a cabo pruebas de "estrés" simulando avances rápidos de tiempo para verificar que los cálculos de BigInteger no provocan desbordamientos de memoria ni ralentizaciones en la interfaz gráfica (ANR - Application Not Responding). Los resultados confirman que la arquitectura MVC separa correctamente el cálculo de la visualización, manteniendo la fluidez.

6. Conclusiones y Líneas futuras

El proyecto demuestra que es posible crear un juego con profundidad estratégica y persistencia en la nube utilizando herramientas gratuitas de Google y un desarrollo nativo sólido. Se ha logrado integrar con éxito una lógica matemática compleja con una interfaz accesible.

Modelo de Monetización Híbrido Para garantizar la sostenibilidad del servidor y el mantenimiento futuro, se ha proyectado un modelo de negocio "Freemium" respetuoso con el usuario:

1. **Anuncios Bonificados (Rewarded Ads):** No se utilizan banners intrusivos. El jugador puede elegir ver un video publicitario voluntariamente para duplicar su producción *offline* al volver al juego.
2. **Micropagos (IAP):** Venta de "Moneda de Prestigio" para usuarios que deseen acelerar el proceso de *Rebirth*, sin bloquear contenido exclusivo detrás de barreras de pago.

Líneas futuras:

- **Eventos Globales:** Añadir un sistema de eventos gestionados desde la nube (ej: "Tormenta Solar: +20% energía solar durante 2h").
- **Leaderboards:** Implementar un ranking de jugadores mediante Firebase para fomentar la competitividad.
- **Expansión de Contenido:** Expandir el árbol de tecnologías hasta niveles de ciencia ficción avanzada y añadir logros desbloqueables en Google Play Games.
- **Multi-idioma:** Ingles, Español, Aleman, etc..

7. Anexos

7.1 Estructura de clases y responsabilidades

Model (Capa de datos):

- **Estructura**: Define propiedades de cada generador (nombre, nivel, costo, produccion)
- **ProgresoUsuario:** Capsula el estado completo del jugador
- **ViewModel** (Logica de negocios):
- **EnergiaViewModel:** Mantiene lista de estructuras, calcula energia, maneja mejoras. Expone LiveData para cada valor observable (energia, energiaPorSegundo, tiempo, taps)

-View (Presentacion):

- **HomeFragment:** Muestra energia actual, barra de progreso, boton de tap manual
- **TiendaFragment:** RecyclerView de estructuras con EstructuraAdapter
- **AjustesFragment**: Opciones de usuario, reset, creditos

-Activity:

- **MainActivity**: Orquesta el ciclo de vida, inicia EnergiaService, maneja navegacion bottom nav
- **LoginActivity:** Flujo de autenticacion con Google

8. Bibliografía y webgrafía

- Documentación de Android Developers.
- Guías oficiales de Firebase Realtime Database.
- Manual de referencia de Java (Oracle).

9. Agradecimientos